

COS 201 LABS ASSESSMENT

THE ORBITAL TEAM

PARTICIPANTS INFORMATION

Student Name	Matric Number	Student ID
Akinnusi Success Joseph	2024/B/CSC/0186	30136092
Anietie Udoh	2024/B/CYB/0402	30163323
Batey Enow	2024/B/CSC/0274	30155074
Abdulmalik Ibrahim	2024/B/SENG/0217	30144113
Ahwe Divine	2024/B/CSC/0206	30125667
James Godswill	2024/B/CYB/0285	30145780
Anjolaiya Oladapo	2024/B/CSC/0211	30151708

Project Title: Student Record System

Create a simple Student Record System in C that allows users to manage student information. The program should include the following functionality:

1. Initial Setup:

- Create a new C project using JetBrains Clion (or your preferred development environment).
- Write a program that displays a welcome message and prompts the user to enter their name. The program should greet the user by their name.

2. Data Storage:

- Implement storage for student information, including name, roll number, and marks, using appropriate data types.

3. Input and Output:

- Add a feature to input a student's marks and display whether the student has passed or failed based on a passing threshold (e.g., marks above 40 are considered passing).

4. Student Records Management:

- Implement a loop that allows the user to input information for multiple students until the user decides to exit.
- Create an array to store information for multiple students, with the ability to add, remove, and modify student records.
- Implement functions for adding, modifying, and displaying student records. Ensure these functions take appropriate arguments and return relevant information.

5. Memory Management:

- Use dynamic memory allocation for student records and ensure memory is freed when no longer needed.

6. File Operations:

- Implement the ability to save student records to a text file and load student records from a text file, storing information such as name, roll number, and marks.

7. Search Functionality:

- Allow users to search for a student by their roll number and display the relevant information.

8. Calculations and Sorting:

- Calculate and display the average marks for all students.
- Provide an option to sort student records based on marks in ascending or descending order.

Project Solution

```
/*
 * COS 201 - Student Record System
 */
#include <stdio.h> // For standard input/output functions
#include <stdlib.h> // For dynamic memory allocation
#include <string.h> // For string functions
// 2. Data Storage: Struct to hold student info
typedef struct {
    char name[100];
    int rollNumber;
    float marks;
} Student;
/* --- Global Variables --- */
// Using a pointer for a dynamic array (meets "array" and "dynamic" requirements)
Student *students = NULL;
int studentCount = 0;
const char *FILENAME = "COS_102_Student_Records.txt";
/* --- Function Prototypes --- */
// These declare all the functions we used
void printMenu();
void addStudent();
void modifyStudent();
void displayStudents();
void searchStudentByRoll();
void saveToFile();
void loadFromFile();
void calculateAverageMarks();
void sortStudentRecords();
int compareMarksAsc(const void *a, const void *b);
int compareMarksDesc(const void *a, const void *b);
void clearInputBuffer();
```

```

/* --- Function Implementations --- */
/**
 * @brief Displays the main menu.
 */
void printMenu() {
    printf("\n--- Student Record System Menu ---\n");
    printf("1. Add Student\n");
    printf("2. Remove Student\n");
    printf("3. Modify Student Record\n");
    printf("4. Display All Students\n");
    printf("5. Search for Student (by Roll No)\n");
    printf("6. Calculate Average Marks\n");
    printf("7. Sort Students by Marks\n");
    printf("8. Save Records to File\n");
    printf("0. Exit\n");
    printf("Enter your choice: ");
}
/**
 * @brief Clears the standard input buffer.
 * Used after scanf to prevent issues with subsequent fgets calls.
 */
void clearInputBuffer() {
    int c;
    while ((c = getchar()) != '\n' && c != EOF);
}
/**
 * @brief 4. Adds a new student record.
 * Uses 5. Dynamic Memory Allocation (realloc).
 */
void addStudent() {
    // 5. Memory Management: Allocate space for one more student
    Student *newStudentList = realloc(students, (studentCount + 1) * sizeof(Student));
    if (newStudentList == NULL) {
        printf("Error: Memory allocation failed!\n");
    }
}

```

```

        return;
    }
    students = newStudentList; // Point to the new (or resized) block of memory
    // Get details for the new student
    Student *newStudent = &students[studentCount]; // Point to the new slot
    printf("Enter Student Name: ");
    fgets(newStudent->name, sizeof(newStudent->name), stdin);
    newStudent->name[strcspn(newStudent->name, "\n")] = 0;
    printf("Enter Roll Number: ");
    while (scanf("%d", &newStudent->rollNumber) != 1) {
        printf("Invalid input. Please enter a number for Roll No: ");
        clearInputBuffer();
    }
    clearInputBuffer(); // Clear buffer after scanf
    printf("Enter Marks: ");
    while (scanf("%f", &newStudent->marks) != 1) {
        printf("Invalid input. Please enter a number for Marks: ");
        clearInputBuffer();
    }
    clearInputBuffer(); // Clear buffer after scanf
    studentCount++;
    printf("Student added successfully.\n");
}

// Function to remove a student
void removeStudent() {
    if (studentCount == 0) {
        printf("No student records to remove.\n");
        return;
    }

    int rollToRemove;
    printf("Enter Roll Number of student to remove: ");
    if (scanf("%d", &rollToRemove) != 1) {
        printf("Invalid Roll Number.\n");
    }
}

```

```

        clearInputBuffer();
        return;
    }
    clearInputBuffer();
    int foundIndex = -1;
    for (int i = 0; i < studentCount; i++) {
        if (students[i].rollNumber == rollToRemove) {
            foundIndex = i;
            break;
        }
    }
    if (foundIndex != -1) {
        // Shift remaining students down
        for (int i = foundIndex; i < studentCount - 1; i++) {
            students[i] = students[i + 1];
        }
        studentCount--;
        // Resize the array
        Student *newStudentList = realloc(students, studentCount * sizeof(Student));
        if (newStudentList != NULL || studentCount == 0) { // realloc can return NULL when
size is 0
            students = newStudentList;
        }
        printf("Student with Roll Number %d removed successfully.\n", rollToRemove);
    } else {
        printf("Student with Roll Number %d not found.\n", rollToRemove);
    }
}

/**
 * @brief 4. Modifies an existing student's record found by roll number.
 */
void modifyStudent() {
    if (studentCount == 0) {
        printf("No student records to modify.\n");
    }
}

```

```

    return;
}

int rollToModify;
printf("Enter Roll Number of student to modify: ");
if (scanf("%d", &rollToModify) != 1) {
    printf("Invalid Roll Number.\n");
    clearInputBuffer();
    return;
}
clearInputBuffer();
int foundIndex = -1;
for (int i = 0; i < studentCount; i++) {
    if (students[i].rollNumber == rollToModify) {
        foundIndex = i;
        break;
    }
}
if (foundIndex != -1) {
    printf("Student Found: %s\n", students[foundIndex].name);
    printf("Enter new Name (current: %s): ", students[foundIndex].name);
    char buffer[100];
    fgets(buffer, sizeof(buffer), stdin);
    buffer[strcspn(buffer, "\n")] = 0;
    // Only update if user entered something (not just pressed Enter)
    if (strlen(buffer) > 0) {
        strcpy(students[foundIndex].name, buffer);
    }
    printf("Enter new Marks (current: %.2f): ", students[foundIndex].marks);
    // Using fgets+atof is safer for optional input
    fgets(buffer, sizeof(buffer), stdin);
    if (strlen(buffer) > 1) { // >1 to check for more than just newline
        students[foundIndex].marks = atof(buffer); // atof converts string to float
    }
}

```

```

        printf("Student record updated successfully.\n");
    } else {
        printf("Student with Roll Number %d not found.\n", rollToModify);
    }
}
/**
 * @brief 4. Displays all student records.
 * Also shows 3. Input/Output (Pass/Fail check).
 */
void displayStudents() {
    if (studentCount == 0) {
        printf("No student records found.\n");
        return;
    }

    printf("\n--- All Student Records (%d) ---\n", studentCount);
    for (int i = 0; i < studentCount; i++) {
        printf("-----\n");
        printf(" Student %d:\n", i + 1);
        printf(" Name:   %s\n", students[i].name);
        printf(" Roll No: %d\n", students[i].rollNumber);
        printf(" Marks:  %.2f\n", students[i].marks);

        // 3. Input and Output: Display Pass/Fail status
        printf(" Status:  %s\n", (students[i].marks > 40) ? "Passed" : "Failed");
    }
    printf("-----\n");
}
/**
 * @brief 7. Search Functionality: Finds and displays a student by roll number.
 */
void searchStudentByRoll() {
    if (studentCount == 0) {
        printf("No student records to search.\n");
    }
}

```

```

        return;
    }
    int rollToSearch;
    printf("Enter Roll Number to search for: ");
    if (scanf("%d", &rollToSearch) != 1) {
        printf("Invalid Roll Number.\n");
        clearInputBuffer();
        return;
    }
    clearInputBuffer();
    for (int i = 0; i < studentCount; i++) {
        if (students[i].rollNumber == rollToSearch) {
            printf("Student Found:\n");
            printf(" Name:   %s\n", students[i].name);
            printf(" Roll No: %d\n", students[i].rollNumber);
            printf(" Marks:  %.2f\n", students[i].marks);
            printf(" Status: %s\n", (students[i].marks > 40) ? "Passed" : "Failed");
            return;
        }
    }

    printf("Student with Roll Number %d not found.\n", rollToSearch);
}
/**
 * @brief 6. File Operations: Saves all records to students.txt.
 */
void saveToFile() {
    FILE *file = fopen(FILENAME, "w");
    if (file == NULL) {
        printf("Error: Could not open file '%s' for writing.\n", FILENAME);
        return;
    }
    // Use a simple CSV (Comma Separated Values) format
    for (int i = 0; i < studentCount; i++) {

```

```

        fprintf(file, "%s,%d,%.2f\n", students[i].name, students[i].rollNumber,
students[i].marks);
    }
    fclose(file);
    printf("Successfully saved %d records to %s\n", studentCount, FILENAME);
}
/**
 * @brief 6. File Operations: Loads records from students.txt.
 */
void loadFromFile() {
    FILE *file = fopen(FILENAME, "r");
    if (file == NULL) {
        printf("No existing records file found ('%s'). Starting fresh.\n", FILENAME);
        return;
    }
    // Clear any existing in-memory records
    free(students);
    students = NULL;
    studentCount = 0;
    char line[256];
    while (fgets(line, sizeof(line), file)) {
        // 5. Memory Management: Grow the array for each line in the file
        Student *newStudentList = realloc(students, (studentCount + 1) * sizeof(Student));
        if (newStudentList == NULL) {
            printf("Error: Memory allocation failed during file load!\n");
            fclose(file);
            return;
        }
        students = newStudentList;
        // Parse the CSV line
        // strtok modifies the string, so we use the 'line' buffer
        char *name = strtok(line, ",");
        char *rollStr = strtok(NULL, ",");
        char *marksStr = strtok(NULL, "\n");
    }
}

```

```

        if (name && rollStr && marksStr) {
            strcpy(students[studentCount].name, name);
            students[studentCount].rollNumber = atoi(rollStr); // Convert string to int
            students[studentCount].marks = atof(marksStr); // Convert string to float
            studentCount++;
        }
    }
    fclose(file);
    printf("Successfully loaded %d records from %s\n", studentCount, FILENAME);
}

/**
 * @brief 8. Calculations: Calculates and displays the average mark.
 */
void calculateAverageMarks() {
    if (studentCount == 0) {
        printf("No students in the system. Cannot calculate average.\n");
        return;
    }
    float totalMarks = 0;
    for (int i = 0; i < studentCount; i++) {
        totalMarks += students[i].marks;
    }
    float average = totalMarks / studentCount;
    printf("Average mark for %d students: %.2f\n", studentCount, average);
}

/**
 * @brief 8. Sorting: Comparison function for qsort (Ascending).
 */
int compareMarksAsc(const void *a, const void *b) {
    Student *s1 = (Student *)a;
    Student *s2 = (Student *)b;
    if (s1->marks < s2->marks) return -1;
    if (s1->marks > s2->marks) return 1;
    return 0;
}

```

```

}
/**
 * @brief 8. Sorting: Comparison function for qsort (Descending).
 */
int compareMarksDesc(const void *a, const void *b) {
    Student *s1 = (Student *)a;
    Student *s2 = (Student *)b;
    if (s1->marks > s2->marks) return -1;
    if (s1->marks < s2->marks) return 1;
    return 0;
}
/**
 * @brief 8. Sorting: Sorts students by marks (ascending or descending).
 */
void sortStudentRecords() {
    if (studentCount < 2) {
        printf("Not enough students to sort (need at least 2).\n");
        return;
    }
    char order;
    printf("Sort in (A)scending or (D)escending order? (A/D): ");
    scanf(" %c", &order); // Note the space to consume whitespace
    clearInputBuffer();
    if (order == 'A' || order == 'a') {
        qsort(students, studentCount, sizeof(Student), compareMarksAsc);
        printf("Students sorted by marks (Ascending).\n");
    } else if (order == 'D' || order == 'd') {
        qsort(students, studentCount, sizeof(Student), compareMarksDesc);
        printf("Students sorted by marks (Descending).\n");
    } else {
        printf("Invalid choice. No sorting performed.\n");
        return;
    }
    // Display the newly sorted list

```

```

    displayStudents();
}
/* --- Main Function --- */
int main() {
    // 1. Initial Setup: Welcome message and user greeting
    char userName[50];
    printf("Welcome to the Student Record System!\n");
    printf("Please enter your name: ");
    fgets(userName, sizeof(userName), stdin);
    userName[strcspn(userName, "\n")] = 0; // Remove the trailing newline from fgets
    printf("\nHello, %s! Let's manage some student records.\n", userName);
    // 6. File Operations: Load existing records on startup
    loadFromFile();
    int choice;
    // 4. Student Records Management: Main loop
    do {
        printMenu();
        if (scanf("%d", &choice) != 1) {
            printf("Invalid input. Please enter a number.\n");
            clearInputBuffer();
            choice = -1; // Set to invalid choice to loop again
            continue;
        }
        clearInputBuffer(); // Clear the newline after scanf
        switch (choice) {
            case 1: addStudent(); break;
            case 2: removeStudent(); break;
            case 3: modifyStudent(); break;
            case 4: displayStudents(); break;
            case 5: searchStudentByRoll(); break;
            case 6: calculateAverageMarks(); break;
            case 7: sortStudentRecords(); break;
            case 8: saveToFile(); break;
            case 0:

```

```
        printf("Saving records and exiting...\n");
        saveToFile(); // Auto-save on exit
        // 5. Memory Management: Free allocated memory
        free(students);
        printf("Goodbye!\n");
        break;
    default:
        printf("Invalid choice. Please enter a number between 0 and 7.\n");
    }
} while (choice != 0);
return 0;
}
```